ЗВІТ

з лабораторної роботи № 4

з дисципліни «Проектування, розробка і реалізація криптографічних систем»

на тему: «Дослідження систем захисту захищених месенджерів»

Виконала:
студентка групи ФІ-52мн
Балацька Вікторія

Київ — 2026

1. Мета роботи

Дослідження принципів побудови систем захищеного обміну повідомленнями, механізмів наскрізного шифрування, моделей загроз та особливостей реалізації криптографічного захисту в сучасних месенджерах.

2. Відповіді на питання щодо принципів наскрізного шифрування

2.1. TLS і наскрізне шифрування (E2EE)

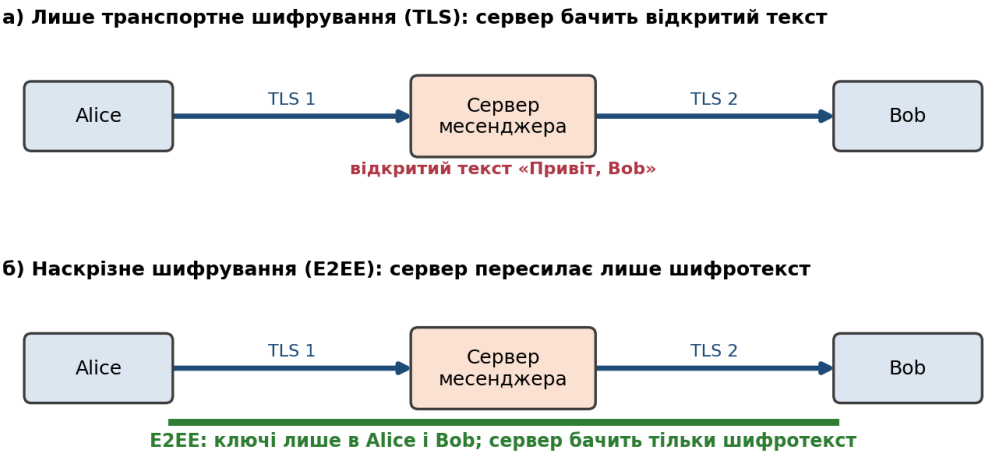


Рисунок 1 — Транспортне та наскрізне шифрування

TLS захищає окремий «перегін» — канал між клієнтом і сервером. На сервері TLS-сесія завершується, і повідомлення стає доступним у відкритому вигляді: оператор, адміністратор, зловмисник, що зламав сервер, або державний орган із запитом можуть його прочитати. Модель довіри — «довіряємо серверу». **E2EE** шифрує повідомлення на пристрої відправника ключем, який мають лише кінцеві пристрої учасників; сервер лише зберігає і пересилає шифротекст і не може його розшифрувати чи непомітно змінити. Модель довіри — сервер вважається недовіреним (а TLS залишається додатковим захистом метаданих у каналі). Межа E2EE: вона захищає зміст, але не метадані (хто, коли, з ким, звідки) і не самі кінцеві пристрої.

2.2. Довгострокові ключі ідентифікації та сеансові ключі

Ключ ідентифікації (identity key, IK) — довгострокова пара ключів (у Signal — Curve25519/Ed25519), що генерується на пристрої й прив'язує криптографічну особу користувача до облікового запису. Використовується для автентифікації: підпису попередніх ключів (signed prekey), участі в DH-обчисленнях під час встановлення сеансу, обчислення safety number. Його зміна (перевстановлення застосунку або атака) видима співрозмовнику. **Сеансові ключі** (кореневий, ланцюгові, ключі повідомлень) короткочасні: виводяться під час встановлення сеансу і постійно оновлюються. Вони шифрують повідомлення. Поділ ролей дає змогу, з одного боку, мати стабільну

перевірювану ідентичність, а з іншого — обмежити шкоду від компрометації: крадіжка ключа одного повідомлення не розкриває інших, а IK сам по собі не розшифровує трафік (без ефемерних ключів).

2.3. Формування спільного секрету через недовірений сервер

Використовується протокол узгодження ключів Діффі–Геллмана на еліптичних кривих (ECDH, X25519): кожна сторона передає лише **відкритий** ключ, а спільний секрет обчислюється локально зі свого закритого ключа і відкритого ключа співрозмовника. Спостерігач, що бачить обидва відкриті ключі, не може обчислити секрет (задача CDH). Для асинхронного месенджера (Bob може бути офлайн) у Signal застосовується **X3DH/PQXDH**: Bob заздалегідь публікує на сервері IK, підписаний середньостроковий ключ SPK і набір одноразових ключів OPK (у PQXDH — також постквантовий ключ ML-KEM, підписаний IK). Alice завантажує цей «пакет», виконує 3–4 DH-обчислення (IK_A·SPK_B, EK_A·IK_B, EK_A·SPK_B, EK_A·OPK_B) і інкапсуляцію ML-KEM та отримує спільний секрет через KDF. Сервер знає лише відкриті ключі. **Але** ECDH сам не захищає від підміни ключів сервером — для цього потрібні підписи ключем ідентифікації і перевірка IK користувачами (п. 2.8).

2.4. Forward Secrecy

Forward Secrecy (пряма секретність) — властивість, за якої компрометація поточних ключів (зокрема довгострокового IK або поточного стану сеансу) **не дозволяє розшифрувати раніше передані повідомлення**. Досягається тим, що ключі повідомлень виводяться односпрямованою KDF і видаляються одразу після використання, а сеанси базуються на ефемерних DH-ключах. Для месенджерів це критично: повідомлення перехоплюються і зберігаються операторами, провайдерами, спецслужбами роками, а пристрій рано чи пізно може бути вкрадено, вилучено або заражено. FS гарантує, що такий майбутній інцидент не розкриє всієї історії листування, записаної раніше в мережі.

2.5. Post-Compromise Security

Post-Compromise Security (PCS, «самовідновлення», future secrecy) — здатність протоколу **відновити конфіденційність після компрометації** стану: якщо зловмисник викрав поточні сеансові ключі, але втратив доступ до пристрою (пасивний після крадіжки), то після кількох обмінів повідомленнями нові ключі знову стануть йому недоступними. Досягається введенням у ключовий розклад нових незалежних секретів — свіжих ефемерних DH-ключів, закриті частини яких атакувальник не бачив. **Різниця:** FS захищає **минуле** від компрометації в майбутньому; PCS захищає **майбутнє** після компрометації в минулому. FS забезпечує симетричний ratchet (одноразові ключі і їх видалення), PCS — лише асиметричний

(DH-) ratchet: симетрична KDF сама по собі не «вилікує» стан, бо атакувальник, який знає ланцюговий ключ, обчислить і всі наступні.

2.6. Чому ключі змінюються в процесі листування

- Щоб забезпечити FS: кожне повідомлення шифрується власним ключем, який після використання знищується.
- Щоб забезпечити PCS: нові DH-ключі обмежують «вікно» компрометації кількома повідомленнями.
- Щоб обмежити обсяг даних, зашифрованих одним ключем, і наслідки витоку одного ключа (ключ повідомлення розкриває лише одне повідомлення).
- Щоб впоратися з асинхронністю і втратою/перестановкою повідомлень: кожне має свій номер у ланцюгу і може бути розшифроване незалежно.

2.7. Механізм Double Ratchet

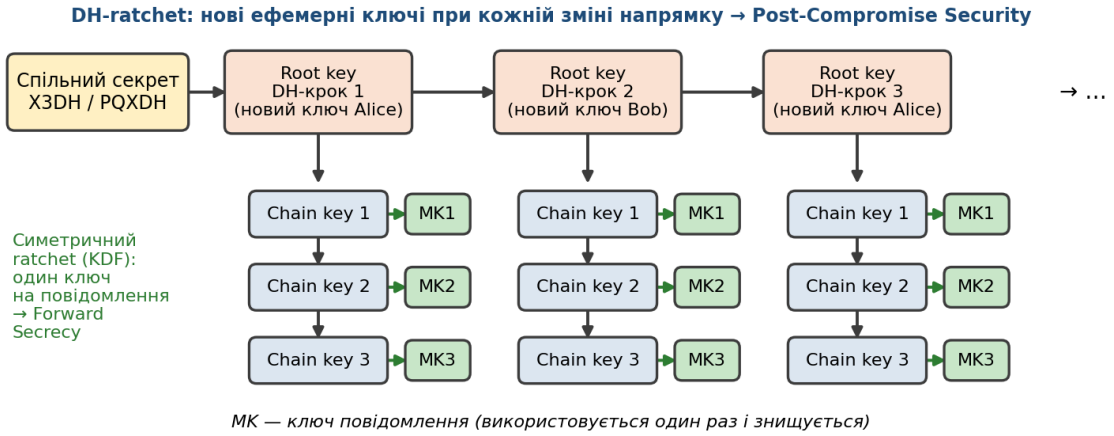


Рисунок 2 — Концептуальна схема Double Ratchet

Double Ratchet (Perrin, Marlinspike, 2016) вирішує задачу **безперервного оновлення ключів в асинхронному листуванні** так, щоб одночасно отримати FS і PCS без додаткових раундів взаємодії. Він поєднує два «храповики» — механізми, що рухаються лише вперед:

- **Симетричний ratchet (ланцюг KDF):** із ланцюгового ключа виводиться ключ повідомлення і наступний ланцюговий ключ; попередні видаляються. Кожне повідомлення має унікальний ключ, минулі ключі відновити неможливо → FS.
- **DH-ratchet:** при кожній зміні напрямку листування сторона генерує нову ефемерну пару ключів і вкладає відкриту частину в заголовок повідомлення. Результат нового DH «підмішується» в кореневий ключ, з якого починаються нові ланцюги → PCS.

Протокол використовується в Signal, WhatsApp, Viber, Facebook Messenger, Skype (private conversations) і сприймається як еталон для асинхронного E2EE. Навчальна модель Double Ratchet, реалізована в межах цієї роботи, наведена в п. 4.3.

2.8. Перевірка співрозмовника: security number, fingerprint, QR-код

Сервер месенджера розподіляє відкриті ключі користувачів, тому скомпрометований або зловмисний сервер може видати Alice **свій** ключ замість ключа Bob і непомітно ретранслювати листування (MITM) — шифрування при цьому «працює». Щоб цього не сталося, користувачі порівнюють позасмуговим (out-of-band) каналом похідну від обох ключів ідентифікації: **safety number** Signal / **security code** WhatsApp (60 цифр), QR-код для сканування при особистій зустрічі, **fingerprint** або візуалізацію ключа (Telegram — зображення й емодзі), «довірені контакти» Viber. Збіг означає, що обидва пристрої бачать однакові ключі й посередника немає. Сповіщення про зміну safety number сигналізує про перевстановлення застосунку або можливу атаку. Прикладом автоматизованої перевірки є **key transparency** (WhatsApp, 2023): публічний журнал ключів, що дозволяє виявити, якщо сервер показує різним користувачам різні ключі.

3. Порівняння систем захищеного обміну повідомленнями

Критерій	Signal	WhatsApp	Telegram	Viber
E2EE в особистих чатах	Так: Signal Protocol (PQXDH + Double Ratchet; з 2025 р. — Triple Ratchet з SPQR)	Так: Signal Protocol (X3DH + Double Ratchet)	Лише в «секретних чатах»: MTProto 2.0 (DH-2048, AES-256-IGE, SHA-256); звичайні «хмарні» чати — шифрування клієнт–сервер, ключі в Telegram	Так: власна реалізація Double Ratchet (Curve25519, Salsa20, HMAC-SHA256)
E2EE за замовчуванням	Так, для всіх чатів і дзвінків, вимкнути неможливо	Так, для всіх особистих і групових чатів і дзвінків	Ні — потрібно вручну створити секретний чат; лише мобільні пристрої, не синхронізується	Так — для 1:1 чатів, груп і 1:1 дзвінків; не для спільнот, каналів, ботів, групових дзвінків, Viber Out
Групові чати	E2EE (Sender Keys); склад групи прихований від сервера (анонімні облікові дані zkgroup)	E2EE (Sender Keys: ключ відправника, розподілений через попарні сесії); склад групи відомий серверу	Без E2EE (хмарні групи до 200 тис. учасників, сервер має доступ до вмісту)	E2EE для груп: спільний симетричний ключ групи, що розподіляється через попарні сесії; спільноти — без E2EE
Голосові й відеодзвінки	E2EE 1:1 і групові (SRTP, ключі з Signal Protocol; SFU пересилає зашифровані кадри)	E2EE 1:1 і групові	E2EE 1:1 (перевірка 4 емодзі); з 2025 р. — E2EE групові дзвінки до 200 учасників з блокчейн-подібним журналом учасників	E2EE 1:1 (DH на ефемерних ключах, Salsa20 для RTP); групові дзвінки — без E2EE
Перевірка автентичності співрозмовника	Safety number (60 цифр) і QR-код; сповіщення про зміну ключа	Security code (60 цифр) і QR; key transparency	Візуалізація ключа секретного чату (зображення/емодзі) ; 4 емодзі у дзвінках	«Довірені контакти»: порівняння ключа під час E2EE-дзвінка
Forward Secrecy	Так (Double Ratchet) + PCS; з PQXDH/SPQR — також проти квантового противника	Так (Double Ratchet) + PCS; для груп — PCS обмежений (оновлення sender key при зміні складу)	Секретні чати: повторне узгодження ключа кожні 100 повідомлень або тиждень (обмежена PFS); хмарні чати — немає E2EE	Для 1:1 — так (ratchet); для груп — обмежено (спільний ключ групи)
Зберігання історії та резервні копії	Історія лише на пристроях; локальна зашифрована копія (Android) і з 2025 р. — зашифровані хмарні копії Secure Backups з ключем відновлення користувача	На пристрої; копії в Google Drive/iCloud — без E2EE за замовчуванням; опційно E2EE-копія (пароль або 64-значний ключ, HSM-сховище ключів)	Хмарні чати зберігаються на серверах Telegram (синхронізація); секретні — лише на пристрої, не потрапляють у хмару	На пристрої; копії в Google Drive/iCloud/Huawei Drive — захищені правилами хмарного провайдера; наскрізний захист копій не задокументовано

Критерій	Signal	WhatsApp	Telegram	Viber
Метадані, доступні оператору	Мінімальні: номер телефону (можна приховати ім’ям користувача), дата реєстрації, дата останнього підключення. Sealed sender приховує відправника	Номер, контакти (за згоди), склад груп, IP, час і частота спілкування, пристрій; частина передається Meta	Для хмарних чатів — усе, включно зі змістом; для секретних — хто, з ким і коли, IP, контакти	Номер, контакти, IP, час і частота, склад груп; зміст спільнот і каналів

Таблиця 1 — Порівняльна характеристика месенджерів

Короткі висновки. Signal реалізує найсуворішу модель: E2EE скрізь і завжди, мінімізацію метаданих і постквантовий захист. WhatsApp використовує той самий криптографічний протокол для вмісту, але модель довіри слабша: широкий збір метаданих і незашифровані за замовчуванням резервні копії. Telegram за замовчуванням є «хмарним» месенджером з довірою до сервера: E2EE — виняток (секретні чати і дзвінки), групи не захищено. Viber надає E2EE за замовчуванням для приватного листування, але з власною реалізацією, спрощеною груповою схемою, винятками для спільнот і групових дзвінків та без задокументованого захисту резервних копій.

Чому наявність шифрування не означає однакового захисту. «Шифрування» в рекламі може означати і TLS до сервера, і справжнє E2EE. Навіть за однакового протоколу рівень захисту визначається: (1) чи ввімкнено E2EE за замовчуванням і для всіх типів спілкування (групи, дзвінки, канали); (2) хто має ключі — лише користувачі чи також оператор; (3) як зберігається історія і резервні копії — «найслабша ланка» може повністю обійти E2EE; (4) які метадані збирає оператор — вони самі по собі розкривають соціальні зв’язки; (5) чи можна перевірити ключі співрозмовника (захист від MITM з боку сервера); (6) відкритість коду та незалежні аудити; (7) властивості протоколу — FS, PCS, постквантова стійкість. Отже, безпеку визначає вся модель довіри, а не наявність шифрування як такого.

4. Аналіз ситуацій порушення безпеки

4.1. Ситуація 1. Компрометація сервера месенджера

Чи може зловмисник прочитати вміст? Ні — за коректного E2EE на сервері є лише шифротексти, а закриті ключі зберігаються тільки на пристроях. Минулі повідомлення, навіть збережені сервером, захищає також FS.

Що він може отримати: метадані (хто, з ким, коли, як часто, розміри повідомлень, IP-адреси, пристрої, склад груп — залежно від месенджера), список

контактів і профілі, відкриті ключі, незашифровані дані (хмарні чати Telegram, спільноти Viber), резервні копії без E2EE, якщо вони на серверах оператора.

Властивості E2EE, що обмежують наслідки: конфіденційність вмісту (ключі лише на пристроях); цілісність і автентичність (AEAD / MAC — сервер не може змінити чи підробити повідомлення); FS (записані раніше шифротексти не розшифруються); у Signal — sealed sender і приватні групи (менше метаданих).

Від чого E2EE не захищає: (1) підміни відкритих ключів (MITM) — якщо користувачі не перевіряють safety number; (2) додавання «привида» до групи — сервер може додати свій пристрій до групи чи облікового запису, якщо клієнти не показують змін складу; (3) розповсюдження шкідливого оновлення клієнта (якщо сервер пов'язаний з каналом оновлень); (4) аналізу метаданих і блокування доставки (DoS, цензура); (5) відмови в обслуговуванні, видалення облікових записів; (6) перехоплення нових реєстрацій (реєстрація облікового запису жертви на новому пристрої через перехоплений SMS-код) — захищає PIN реєстрації (Registration Lock).

4.2. Ситуація 2. Компрометація кінцевого пристрою

E2EE не захищає в цьому випадку: розблокований смартфон — це кінцева точка, де повідомлення за визначенням розшифровані. Зловмисник може прочитати всю збережену історію, надсилати повідомлення від імені власника, скопіювати ключі і стан сеансів (і далі пасивно читати нове листування — до «самовідновлення», п. 4.3), відскануванням QR прив'язати свій пристрій (веб/настільна версія) як «пов'язаний пристрій», а також встановити шпигунське ПЗ (Pegasus-подібні засоби зчитують екран і клавіатуру до шифрування).

Чому захист пристрою — окрема складова: криптографія захищає дані «у дорозі» та від сервера, а на пристрої діють механізми ОС і застосунку: блокування екрана, шифрування сховища (File-Based Encryption, Secure Enclave/StrongBox для ключів), пароль на сам застосунок, зникаючі повідомлення (менше історії для крадіжки), контроль списку пов'язаних пристроїв, Registration Lock, заборона знімків екрана, своєчасні оновлення ОС. Модель загроз E2EE прямо припускає, що кінцеві точки довірені; якщо це припущення порушено, жоден протокол не допоможе.

4.3. Ситуація 3. Компрометація ключового матеріалу

Можливості зловмисника залежать від того, **що саме** викрадено і від протоколу:

- **Один ключ повідомлення (MK)** — розшифровується лише це повідомлення.
- **Поточний ланцюговий ключ** — розшифровуються наступні повідомлення цього ланцюга до найближчого DH-кроку; попередні — ні (KDF односпрямована).

- **Повний стан сеансу** (кореневий ключ + закритий DH-ключ) — пасивний атакувальник читає поточний ланцюг і відповідь співрозмовника (обчислює DH зі старим закритим ключем), але втрачає доступ, щойно власник викраденого стану згенерує новий DH-ключ.
- **Довгостроковий IK** — не розкриває минулих повідомлень (FS), але дозволяє видавати себе за власника в **нових** сеансах (активний MITM), тому IK потребує захисту апаратним сховищем.
- **Протокол без ratchet** (напр., статичний ключ) — розкривається вся історія і все майбутнє листування.

Для перевірки цих тверджень реалізовано навчальну модель Double Ratchet мовою Python (X25519, HKDF-SHA256, HMAC-SHA256, AES-256-GCM; бібліотека `cryptography` 46.0.7). Зловмисник копіює повний стан пристрою Alice після повідомлення A2, а далі лише пасивно перехоплює шифротексти:

```
Стан Alice викрадено після повідомлення A2.

Повідомлення                          Спроба зловмисника
A1: привіт                            не вдалося (ключі видалено, KDF односпрямована — FS)
A2: номер рахунку 1234                не вдалося (ключі видалено, KDF односпрямована — FS)
A3: пароль від Wi-Fi                  РОЗШИФРОВАНО: A3: пароль від Wi-Fi
B1: дякую                             РОЗШИФРОВАНО: B1: дякую
B2: зустрінемось о 18:00              РОЗШИФРОВАНО: B2: зустрінемось о 18:00
A4: гаразд                            не вдалося (новий DH-ключ Alice — PCS)
A5: нова адреса — вул. Садова, 5      не вдалося (новий DH-ключ Alice — PCS)
```

Лістинг 1 — Демонстрація FS і PCS (ratchet_demo.py)

Роль механізмів: Forward Secrecy захистила A1–A2 (ключі видалено, KDF незворотна). **Зміна сеансових ключів** обмежила вікно компрометації: розкрито лише повідомлення до наступного DH-кроку (A3, B1, B2). **Post-Compromise Security:** щойно Alice згенерувала новий ефемерний DH-ключ (A4), атакувальник, який не контролює пристрій постійно, втратив доступ. Обмеження: PCS не працює проти активного атакувальника, що залишається на пристрої або виконує MITM з викраденим IK; у групах із Sender Keys самовідновлення відбувається лише після оновлення ключа відправника.

4.4. Ситуація 4. Атака «людина посередині»

Чому зашифроване з'єднання не підтверджує особу. Alice отримує «ключ Bob» від сервера, якому не довіряє. Якщо сервер (або той, хто його контролює) підставить ключ Mallory, Alice встановить абсолютно коректний зашифрований сеанс — але з Mallory. Mallory так само встановить сеанс з Bob і пересилатиме повідомлення, розшифровуючи і, за бажання, змінюючи їх. Шифрування гарантує лише, що дані читає власник **того** ключа, з яким узгоджено сеанс; хто цей власник — вирішується лише автентифікацією ключа. На відміну від TLS, де прив'язку ключа до імені

засвідчує ЦС, у месенджерах немає незалежного засвідчувача — сервер сам розподіляє ключі.

Виявлення MITM через перевірку. Кожен клієнт обчислює safety number / fingerprint як хеш від пари ключів ідентифікації, **які він фактично використовує**. Без атаки обидва бачать однакові IK — числа збігаються. При MITM Alice використовує IK_Mallory замість IK_Bob, а Bob — IK_Mallory замість IK_Alice, тому числа відрізняються. Порівняння потрібно виконати каналом, який Mallory не контролює: сканування QR при зустрічі, телефонний/відеодзвінок з голосом співрозмовника, інший месенджер. Результат моделювання (спрощене обчислення за зразком Signal — 60 цифр, групи по 5):

```
Safety number без атаки:
  Alice (IK_A+IK_B): 21808 66717 77460 78438 74569 25302 63468 81048 74831 55964 66406 47236
  Bob   (IK_A+IK_B): 21808 66717 77460 78438 74569 25302 63468 81048 74831 55964 66406 47236
Сервер підмінив ключі (MITM Mallory):
  Alice (IK_A+IK_M): 46395 74107 80373 56425 71036 14846 65184 70167 75667 13933 94140 22434
  Bob   (IK_M+IK_B): 76323 86675 45342 12016 20309 57739 67584 91479 13351 10419 56005 54118
```

Лістинг 2 — Safety number без атаки та при підміні ключів сервером

Обмеження: більшість користувачів не перевіряє коди, а повідомлення про «зміну коду безпеки» часто ігнорують. Тому месенджери впроваджують **key transparency** — автоматичну перевірку того, що сервер показує всім однакові ключі.

4.5. Ситуація 5. Резервне копіювання повідомлень

Чому E2EE не поширюється на копію. E2EE захищає повідомлення в дорозі між пристроями. Після розшифрування на пристрої історія зберігається локально і для резервного копіювання експортується в хмару (Google Drive, iCloud, сервер месенджера) — це **нова кінцева точка** поза моделлю E2EE. Якщо копія зашифрована ключем, доступним хмарному провайдеру (або не зашифрована окремо), її можуть прочитати провайдер, зловмисник, що отримав доступ до облікового запису Google/Apple, або правоохоронні органи за запитом. Одна незахищена копія будь-якого з учасників розмови розкриває все листування, навіть якщо інший учасник копій не робить.

Вимоги, за яких копія зберігає властивості наскрізного захисту:

- Шифрування на пристрої до вивантаження (client-side encryption) стійким AEAD-алгоритмом.
- Ключ відомий лише користувачеві: випадковий ключ відновлення (напр., 64 символи) або ключ, похідний від пароля через стійку KDF (Argon2/scrypt) з обмеженням спроб на стороні сервера в HSM (як Backup Key Vault у WhatsApp), щоб короткий пароль не можна було перебрати.

- Ні оператор месенджера, ні хмарний провайдер не мають копії ключа; відновлення без ключа неможливе.
- Автентифікація і цілісність копії (виявлення підміни), захист від відкату до старої копії.
- Мінімізація: зникаючі повідомлення не потрапляють до копій, можливість вимкнути копіювання, видалення старих копій.
- Прозорість: користувач бачить, чи ввімкнено E2EE-копіювання (у WhatsApp воно опційне, за замовчуванням вимкнене).

5. Постквантовий захист систем обміну повідомленнями

5.1. Сценарій Harvest Now, Decrypt Later

«Збери зараз — розшифруй пізніше» (HNDL): противник уже сьогодні перехоплює і зберігає зашифрований трафік (включно з рукостисканнями, де передаються відкриті ключі ECDH), розраховуючи розшифрувати його в майбутньому, коли з'явиться криптографічно значущий квантовий комп'ютер (CRQC). Алгоритм Шора розв'язує задачі дискретного логарифма (у тому числі на еліптичних кривих) і факторизації за поліноміальний час, тому з записаних відкритих ключів X25519/RSA можна буде відновити спільні секрети і всі похідні ключі повідомлень.

5.2. Чому загроза актуальна вже зараз

- Для конфіденційності важливий не момент перехоплення, а **строк, протягом якого інформація має залишатися таємною**: дипломатичне, військове, медичне, особисте листування може бути чутливим 10–30 років.
- Якщо строк секретності (X) + час міграції систем (Y) більші за час до появи CRQC (Z), дані вже під загрозою (нерівність Моски: $X + Y > Z$).
- Держави та великі організації мають ресурси для масового збору трафіку, а зберігання дешевшає.
- Міграція протоколів і клієнтів займає роки, тому її потрібно починати до появи загрози; NIST у 2024 р. стандартизував ML-KEM (FIPS 203), а плани переходу (NIST IR 8547) передбачають відмову від класичних алгоритмів з відкритим ключем до 2030–2035 рр.
- Для **автентифікації** (підписи) загроза настає лише з появою CRQC (підробити підпис «у минулому» не можна), тому пріоритет — захист обміну ключами.

5.3. Механізми, що потребують постквантового захисту

Механізм	Чим загрожує квантовий комп'ютер	Пріоритет
Початкове узгодження ключа (X3DH: ECDH X25519)	Відновлення спільного секрету з записаних відкритих ключів → розкриття всього сеансу (HNDL)	Найвищий

Механізм	Чим загрожує квантовий комп'ютер	Пріоритет
DH-ratchet (Double Ratchet)	Кожен DH-крок може бути зламано → FS і PCS втрачаються проти квантового противника	Високий
Узгодження ключів дзвінків, групових ключів, резервних копій на основі ECDH	HNDL для записаних дзвінків і копій	Високий
Підписи (Ed25519/XEdDSA для signed prekey, identity; ECDSA/RSA у TLS)	Підробка ключів і MITM — лише після появи CRQC, в реальному часі	Середній (планова міграція на ML-DSA)
Симетричні алгоритми (AES-256, HMAC-SHA-256, ChaCha20)	Алгоритм Гровера лише вдвічі зменшує стійкість (256 → 128 біт)	Низький — достатньо 256-бітових ключів

Таблиця 2 — Механізми месенджерів і квантові загрози

5.4. Інтеграція постквантових механізмів у Signal

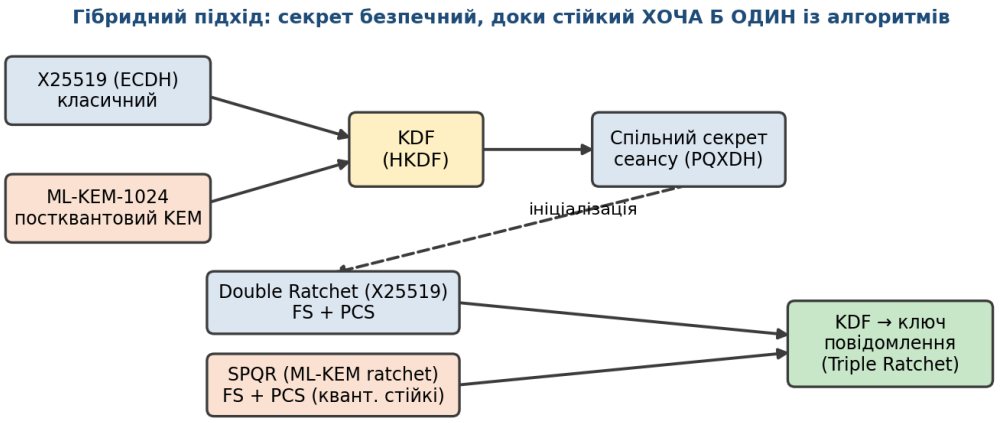


Рисунок 3 — Гібридна постквантова схема Signal (PQXDH + Triple Ratchet)

Етап 1 — PQXDH (вересень 2023 р.). До пакета попередніх ключів Bob додано постквантовий ключ ML-KEM-1024 (раніше CRYSTALS-Kyber), підписаний ключем ідентифікації. Alice, окрім DH-обчислень X3DH, інкапсулює випадковий секрет на цей ключ і передає шифротекст KEM у першому повідомленні. Остаточний секрет — це KDF від конкатенації результатів ECDH **і** секрету ML-KEM. Це закриває HNDL для початкового секрету кожного нового сеансу.

Етап 2 — SPQR і Triple Ratchet (жовтень 2025 р.). PQXDH захищав лише початок сеансу, а подальші DH-кроки залишалися класичними, тому квантовий противник, який зламав би їх, отримав би ключі після компрометації. Signal додав **Sparse Post-Quantum Ratchet (SPQR)** — постквантовий ratchet на основі ML-KEM, що працює паралельно з Double Ratchet. Ключі повідомлень виводяться KDF одночасно з обох храповиків («Triple Ratchet»). Оскільки ключі й шифротексти ML-KEM мають понад 1000 байтів (порівняно з 32 байтами X25519), їх розбивають на фрагменти з кодом стирання (erasure coding) і передають частинами в заголовках звичайних повідомлень — «sparse» ratchet оновлюється рідше, ніж EC-ratchet, але без

додаткових повідомлень. Перехід сумісний зі старими клієнтами, після узгодження понижувальне повернення блокується.

5.5. Додаткова властивість безпеки

Модернізація має забезпечити **конфіденційність проти квантового противника (post-quantum confidentiality), у тому числі щодо записаного сьогодні трафіку**: навіть маючи в майбутньому CRQC, атакувальник не зможе відновити сеансові ключі з перехоплених рукостискань. Разом з SPQR це поширюється і на **FS та PCS проти квантового противника** протягом усього сеансу. Принципова особливість — **гібридність**: секрет залишається захищеним, доки стійкий хоча б один з алгоритмів. Якщо в новому ML-KEM буде знайдено класичну вразливість, захист не гірший за X25519; якщо з'явиться квантовий комп'ютер — захищає ML-KEM. Автентифікація (підписи) поки залишається класичною — це свідомий компроміс, оскільки загроза підробки підпису не має ретроспективного характеру.

6. Висновки

1. Наскрізне шифрування принципово змінює модель довіри порівняно з TLS: сервер стає недовіреним посередником, який бачить лише шифротексти. Ключову роль відіграють поділ на довгострокові ключі ідентифікації (автентифікація) і короткочасні сеансові ключі (шифрування), асинхронне узгодження ключів X3DH/PQXDH та Double Ratchet, що забезпечує Forward Secrecy і Post-Compromise Security.

2. Порівняння показало, що Signal, WhatsApp, Telegram і Viber мають суттєво різні моделі захисту: від E2EE скрізь з мінімізацією метаданих і постквантовим захистом (Signal) до хмарної моделі з довірою до сервера і E2EE лише за окремим бажанням (Telegram). Рівень захисту визначають покриття E2EE, керування ключами, резервні копії, метадані та можливість перевірки ключів, а не сам факт «шифрування».

3. Аналіз ситуацій порушення безпеки визначив межі E2EE: воно захищає вміст від скомпрометованого сервера і обмежує наслідки викрадення ключів, але не захищає від компрометації кінцевого пристрою, аналізу метаданих, MITM без перевірки ключів і незахищених резервних копій. Навчальна модель Double Ratchet підтвердила, що після викрадення стану розкриваються лише повідомлення до наступного DH-кроку, а перевірка safety number виявляє підміну ключів сервером.

4. Загроза Harvest Now, Decrypt Later робить постквантовий захист обміну ключами актуальним уже сьогодні. Signal демонструє практичний шлях — гібридні схеми PQXDH і Triple Ratchet (X25519 + ML-KEM), які зберігають класичну стійкість

і додають конфіденційність, FS і PCS проти квантового противника; цифрові підписи мігруватимуть пізніше.

7. Перелік використаних джерел

1. Marlinspike M., Perrin T. The X3DH Key Agreement Protocol. Signal, 2016. URL: https://signal.org/docs/specifications/x3dh/
2. Perrin T., Marlinspike M. The Double Ratchet Algorithm. Signal, 2016. URL: https://signal.org/docs/specifications/doubleratchet/
3. Kret E., Schmidt R. The PQXDH Key Agreement Protocol. Signal, 2023. URL: https://signal.org/docs/specifications/pqxdh/
4. Signal Blog. Signal Protocol and Post-Quantum Ratchets (SPQR). 02.10.2025. URL: https://signal.org/blog/spqr/
5. Signal Blog. Introducing Signal Secure Backups. 2025. URL: https://signal.org/blog/introducing-secure-backups/
6. WhatsApp Encryption Overview. Technical White Paper. URL: https://www.whatsapp.com/security/WhatsApp-Security-Whitepaper.pdf
7. Meta Engineering. Deploying key transparency at WhatsApp. 2023; How WhatsApp enables end-to-end encrypted backups. 2021. URL: https://engineering.fb.com/
8. Telegram. MTProto Mobile Protocol: End-to-End Encryption, Secret Chats. URL: https://core.telegram.org/api/end-to-end
9. Telegram. E2E Group Calls. URL: https://core.telegram.org/api/end-to-end/group-calls; Telegram Blog, 30.04.2025. URL: https://telegram.org/blog/group-calls-made-easy
10. Viber Encryption Overview. URL: https://www.viber.com/app/uploads/viber-encryption-overview.pdf
11. Viber Support. End-to-end encryption in chats. URL: https://help.viber.com/hc/en-us/articles/8909167863453
12. Privacy International. Guide on Viber Settings and Good Practices. URL: https://privacyinternational.org/guide-step/5615/guide-viber-settings-and-good-practices
13. Cohn-Gordon K. et al. A Formal Security Analysis of the Signal Messaging Protocol. Journal of Cryptology, 2020.
14. NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). 2024.
15. NIST IR 8547 (ipd). Transition to Post-Quantum Cryptography Standards. 2024.
16. Post-Quantum Security. Which Messengers Are Already Quantum-Safe? URL: https://www.postquantumsecurity.org/publications/pqc_messengers.html